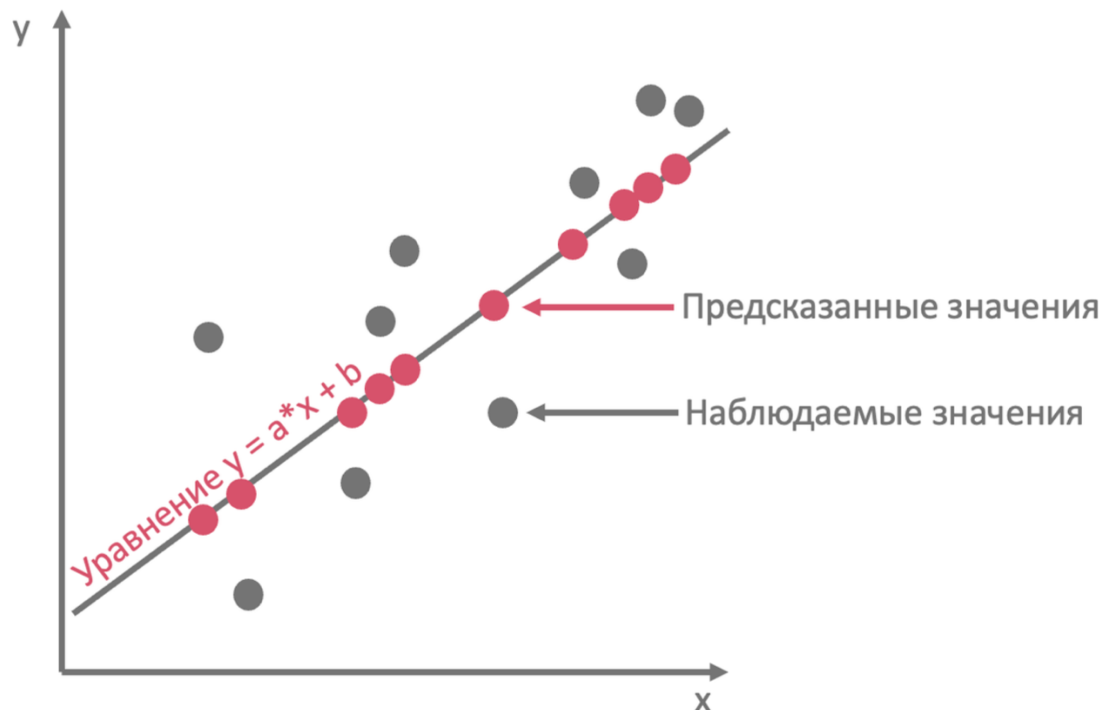


|                        |                                             |
|------------------------|---------------------------------------------|
| ДИСЦИПЛИНА             | Базы данных и анализ<br>промышленных данных |
| ИНСТИТУТ               | Передовая инженерная школа                  |
| КАФЕДРА                | Кафедра передовых технологий                |
| ВИД УЧЕБНОГО МАТЕРИАЛА | Практическое занятие                        |
| ПРЕПОДАВАТЕЛЬ          | Астафьев Р.У.                               |
| СЕМЕСТР                | 2 семестр, 2025 - 2026 гг.                  |

## Практическая работа 7

**Регрессионный анализ данных: методы предпосылки, методы проведения, анализ результатов.**

Регрессия — это статистический метод, который связывает зависимую переменную с одной или несколькими независимыми переменными (строит функцию  $y = a_1 \cdot x_1 + a_2 \cdot x_2 + \dots + a_n \cdot x_n + b$ , по которой можно предсказать значение зависимой переменной  $y$  по независимым переменным  $x_1, x_2, \dots, x_n$ ).



### Основные типы регрессии:

1. Линейная регрессия - метод, связывающий зависимую переменную и независимую переменную линейной функцией. Также распространенным видом линейной регрессии является множественная регрессия, которая связывает зависимую переменную с несколькими независимыми переменными, в таком случае результат модели - линейная комбинация независимых переменных. Коэффициенты для данного метода регрессии подбираются методом наименьших квадратов.

2. Полиномиальная регрессия - метод, связывающий зависимую и независимые переменные полиномиальной функцией, например,  $y = a_1 \cdot x_1 + (a_2)^2 \cdot x_2 + (a_3)^4 \cdot x_3 \dots a_n \cdot x_n + b$ .

3. Логистическая регрессия — это тип регрессии, в которой мы оцениваем наличие связи между дихотомической зависимой переменной и одной или несколькими независимыми переменными

### Предпосылки регрессии

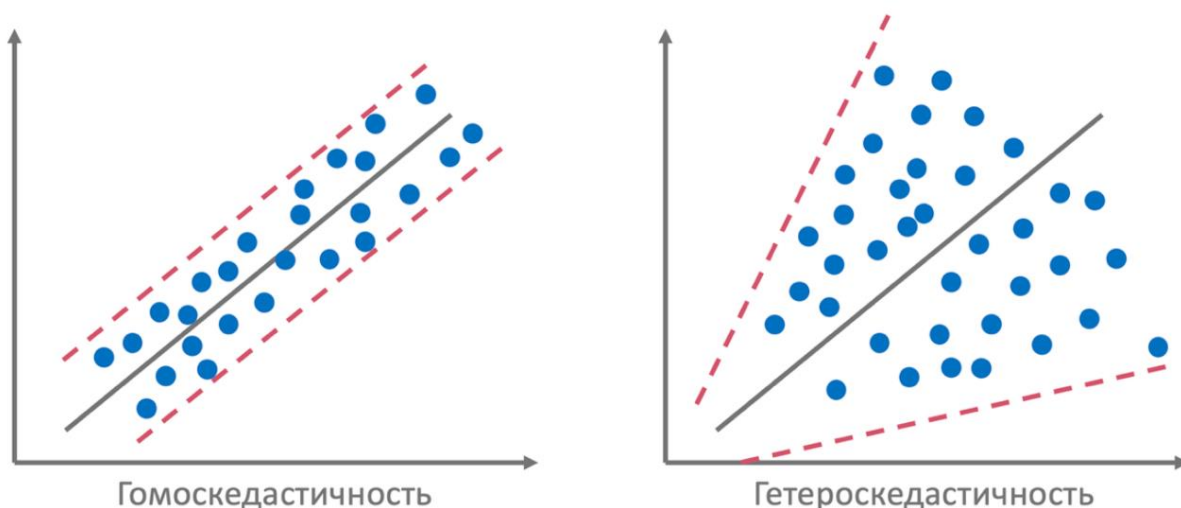
Для корректной работы алгоритмов регрессионных моделей необходимо следить не только за количеством наблюдений (минимум 30), отсутствием выбросов и качеством данных, но и соблюдать следующие предпосылки по теореме Гаусса-Маркова:

1. Модель линейна по параметрам и корректно специфицирована (означает, что выбрана правильная функциональная форма модели и в неё включены необходимые независимые переменные, и не включаются избыточные или нерелевантные переменные)

2. Независимые переменные являются детерминированными и линейно независимыми (независимые переменные не коррелируют и их количество не превосходит число наблюдений). Если независимые переменные коррелируют, то в данных присутствует мультиколлинеарность

3. Математическое ожидание случайных ошибок равно нулю (в данном случае случайная ошибка — это различие между наблюдаемыми и предсказанными значениями. Равенство математического ожидания нулю означает независимость ошибок между собой)

4. Дисперсия случайной ошибки одинакова для всех наблюдений (данное условие означает, что дисперсия случайной ошибки в каждом наблюдении имеет только одно значение). Условие постоянства дисперсии случайной ошибки от номера наблюдения называется гомоскедастичностью, условие непостоянства дисперсии — гетероскедастичностью



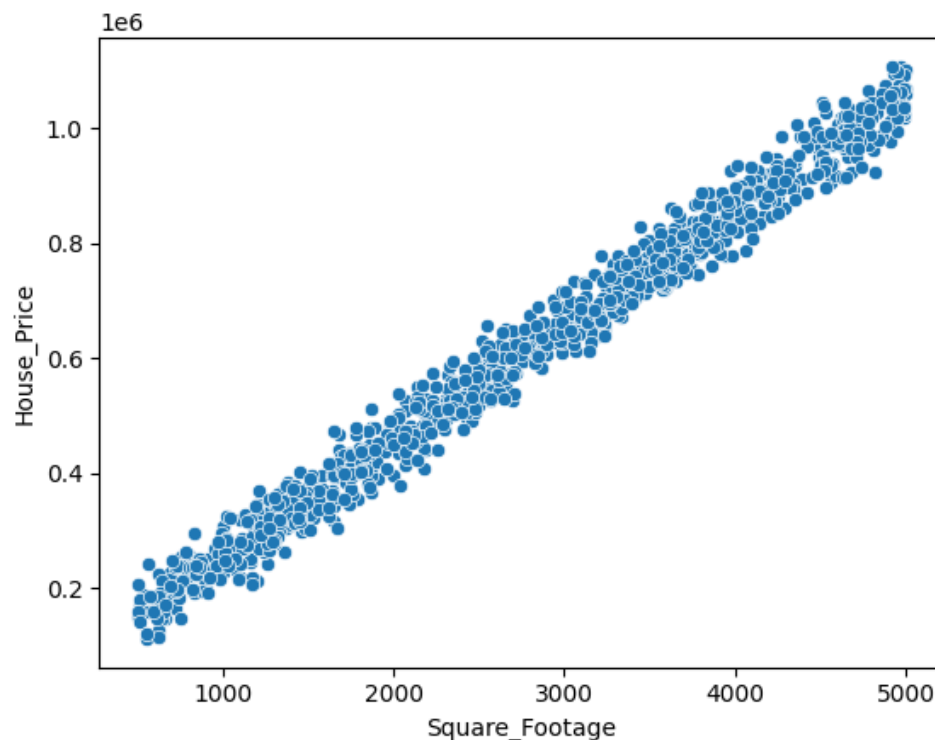
### 5. Случайные ошибки имеют нормальное распределение

```
import pandas as pd # Импортируем библиотеки для работы с данными
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
```

```
df=pd.read_csv('house_price_regression_dataset.csv') # Загружаем датасет
о стоимости недвижимости
# Подробное описание данных:
https://www.kaggle.com/datasets/prokshitha/home-value-insights/data
df.head() # Проверяем загрузку датасета
```

|   | Square_Footage | Num_Bedrooms | Num_Bathrooms | Year_Built | Lot_Size | Garage_Size | Neighborhood_Quality | House_Price  |
|---|----------------|--------------|---------------|------------|----------|-------------|----------------------|--------------|
| 0 | 1360           | 2            | 1             | 1981       | 0.599637 | 0           | 5                    | 2.623829e+05 |
| 1 | 4272           | 3            | 3             | 2016       | 4.753014 | 1           | 6                    | 9.852609e+05 |
| 2 | 3592           | 1            | 2             | 2016       | 3.634823 | 0           | 9                    | 7.779774e+05 |
| 3 | 966            | 1            | 2             | 1977       | 2.730667 | 1           | 8                    | 2.296989e+05 |
| 4 | 4926           | 2            | 1             | 1993       | 4.699073 | 0           | 8                    | 1.041741e+06 |

```
# Построим модель линейной регрессии, где у - стоимость дома, x - площадь
sns.scatterplot(df, x='Square_Footage', y='House_Price') # Выведем
диаграмму рассеяния
```

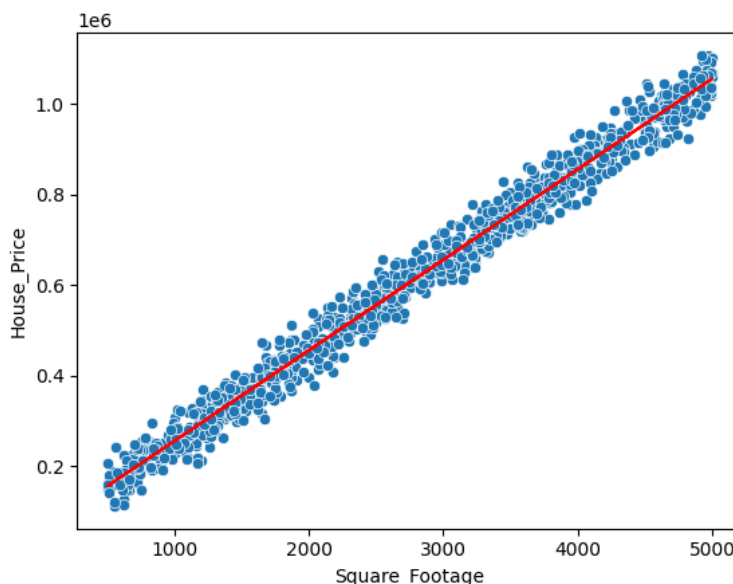


```
from sklearn.linear_model import LinearRegression # Для построения
линейной регрессии используем метод LinearRegression
from sklearn import metrics # Импортируем метрики для оценки качества
модели
import numpy as np # Импортируем библиотеку для работы с массивами

x, y = np.array(df['Square_Footage']).reshape(-1, 1),
np.array(df['House_Price']).reshape(-1, 1) # определяем независимую и
зависимую переменные, метод reshape транспонирует массив
model = LinearRegression() # Объявляем модель
model.fit(x, y) # Обучаем модель
print('Коэффициент при x:', model.coef_, 'Свободный коэффициент:',
model.intercept_) # Выведем полученные коэффициенты
```

Коэффициент при x: [[200.19852991]] Свободный коэффициент: [55217.67316984]

```
sns.scatterplot(df, x='Square_Footage', y='House_Price') # Отообразим на
графике полученную модель
plt.plot(x, [model.intercept_[0] + model.coef_[0] * l for l in x], col-
or='r')
```



```
model.predict(np.array([50, 100, 150, 100]).reshape(-1, 1)) # Также мы
можем предсказать любые значения y по переменной x
array([[65227.59966536],
       [75237.52616087],
       [85247.45265638],
       [75237.52616087]])
```

Качество линейной регрессионной модели оценивается по следующим параметрам:

1. R-квадрат (коэффициент детерминации). Показывает долю дисперсии зависимой переменной, объясненной с помощью регрессионной модели. Он изменяется от 0 до 1: при R больше 0,8 говорят, что модель хорошо объясняет данные, при R больше 0,5 результат является удовлетворительным. Если значение R меньше 0,5, то модель плохая. Также есть скорректированный коэффициент детерминации, который позволяет сравнивать модели с разным количеством переменных

2. MSE - среднеквадратичная ошибка

3. RMSE - корень из среднеквадратичной ошибки

4. MAE - средняя абсолютная ошибка

5. MAPE - средняя абсолютная процентная ошибка

Пункты 2,3,4,5 - абсолютные ошибки, не позволяющие чётко оценить размеры ошибок, поэтому для оценки качества регрессионной модели предпочтительнее использовать относительную метрику R-квадрат

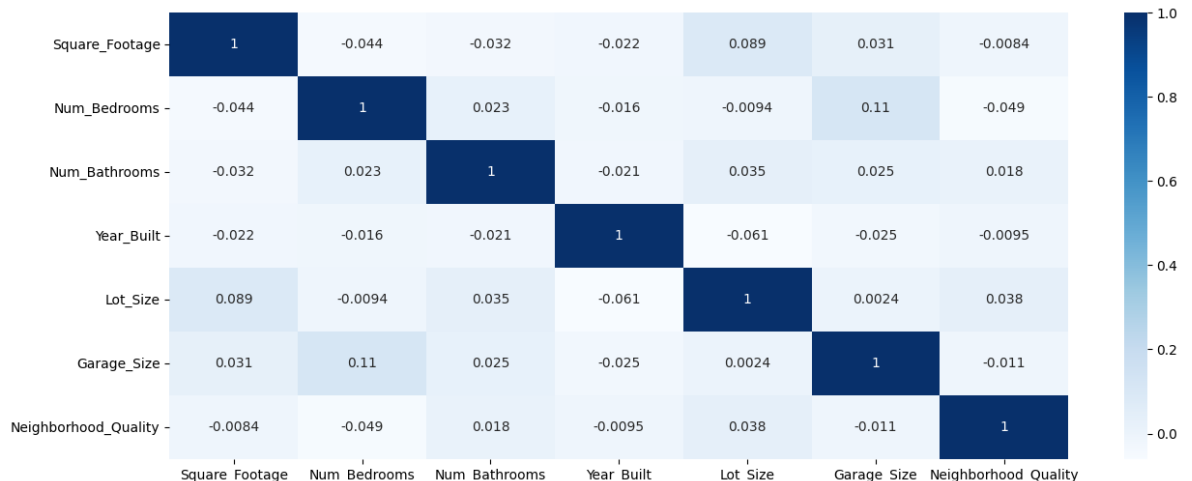
```
# Рассчитаем метрики качества для моделей. Для этого нам необходимо
предсказать все значения y по x для расчета ошибки (|y_настоящее -
y_предсказанное|)
y_predict_values = model.predict(x) # Предсказываем значения
r_square = metrics.r2_score(y, y_predict_values) # Коэффициент
детерминации
mse = metrics.mean_squared_error(y, y_predict_values) ** 0.5 #MSE
mae = metrics.mean_absolute_error(y, y_predict_values) # MAE
print('R-квадрат:', r_square, 'MSE:', mse, 'MAE:', mae)
```

# Значение R-квадрат = 0.88 говорит о хорошем качестве модели

**R-квадрат: 0.9825992634150338 MSE: 33431.903580428225 MAE: 26996.72051668837**

# Построим модель множественной линейной регрессии. Для этого необходимо убедиться в отсутствии мультиколлинеарности у переменных  $x_1, x_2 \dots x_n$   
# Для определения можно отобразить матрицу корреляции или рассчитывать метрику VIF

```
plt.figure(figsize=(15, 6))
sns.heatmap(df[['Square_Footage', 'Num_Bedrooms', 'Num_Bathrooms',
'Year_Built', 'Lot_Size', 'Garage_Size', 'Neighbor-
hood_Quality']].corr(), annot=True, cmap="Blues")
# По графику убеждаемся в отсутствии мультиколлинеарности
```



# Процесс аналогичен построению обычной линейной регрессии, но теперь в  $x$  набор данных из нескольких столбцов

```
x, y = np.array(df[['Square_Footage', 'Num_Bedrooms', 'Num_Bathrooms',
'Year_Built', 'Lot_Size', 'Garage_Size', 'Neighborhood_Quality']]),
np.array(df['House_Price']).reshape(-1, 1)
model = LinearRegression() # Объявляем модель
model.fit(x,y) # Обучаем модель
model.coef_ # Выводим полученные коэффициенты
# Коэффициенты показывают силу и характер влияния независимых переменных
на зависимую и характеризуют степень значимости отдельных переменных
array([[ 199.75641134, 10170.95455485, 8244.53504682, 991.46815331,
14919.32317901, 5158.06375911, 80.61739971]])
```

# Исследуем качество модели

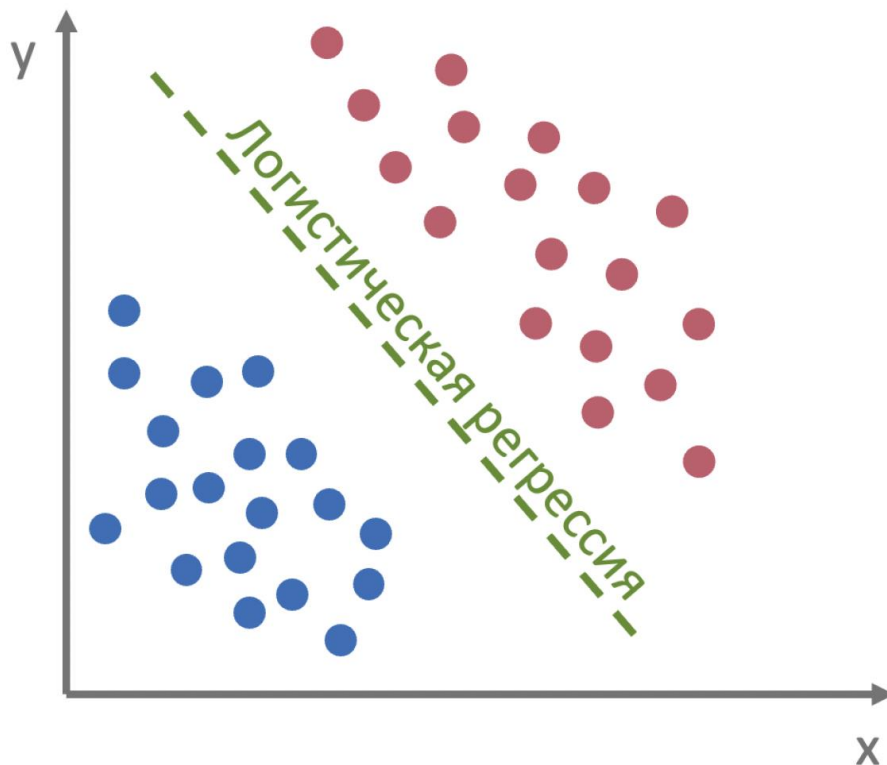
```
y_predict_values = model.predict(x)
r_square = metrics.r2_score(y, y_predict_values)
mse = metrics.mean_squared_error(y, y_predict_values) ** 0.5
mae = metrics.mean_absolute_error(y, y_predict_values)
print('R-квадрат:', r_square, 'MSE:', mse, 'MAE:', mae)
# R-квадрат близок к единице, следовательно модель хорошо описывает
поведение y
```

**R-квадрат: 0.9985171191457451 MSE: 9759.564042435331 MAE: 7740.4300892328665**

## Логистическая регрессия

Предсказывает переменную в интервале  $[0,1]$  при любых значениях независимых переменных (из-за логит преобразования) и рассчитывается значение вероятности (вероятность того, что  $y = 1$  при определённом

значении  $x$ ). Основная идея логистической регрессии заключается не в описании данных при помощи модели, а их разделения.



```
# Для построения логистической регрессии создадим дихотомическую
переменную по стоимости жилья: 1- дорогое жилье, 0 - не дорогое,
разделение будем проводить при помощи медианы
df['Expensive'] = 0
df.loc[df['House_Price']>df['House_Price'].median(), "Expensive"] = 1

from sklearn.linear_model import LogisticRegression # Импортируем
логистическую регрессию
x, y = np.array(df['Square_Footage']).reshape(-1, 1),
np.array(df['Expensive']).reshape(-1, 1) # Определяем переменные
model = LogisticRegression() # Объявляем модель
model.fit(x, y) # Обучаем модель
model.intercept, model.coef # Выводим коэффициенты полученной модели

(array([-28.8287026]), array([[0.01007734]]))
```

Качество логистической регрессии оценивается при помощи метрики accuracy, матрицы ошибок и визуализации

```
# Рассчитаем accuracy
y_predict_values = model.predict(x)
probs = model.predict_proba(x)
metrics.accuracy_score(y, y_predict_values)
```

**0.971**

```
# Построим матрицу ошибок
pd.DataFrame(metrics.confusion_matrix(y, y_predict_values),
             columns = ['Forecast 0', 'Forecast 1'],
             index = ['Actual 0', 'Actual 1'])
```

# По матрице ошибок видим, что алгоритм ошибся 29 раз из 1000: 15 раз он посчитал, что наблюдение относится к классу 1, хотя на самом деле это был класс 0, и 14 раз, наоборот, неверно отнес класс 1 к классу 0.

|          | Forecast 0 | Forecast 1 |
|----------|------------|------------|
| Actual 0 | 485        | 15         |
| Actual 1 | 14         | 486        |

### Задание 1 (2 балла)

1. Скачать данные по ссылке

<https://www.kaggle.com/datasets/stealthtechnologies/predict-the-age-of-a-fossil>;

2. Построить регрессионную модель, предсказывающую возраст ископаемого с значением R-квадрат не ниже 0,8;

3. Сгенерировать 10 значений для независимых переменных и на основании полученной модели определить возраст ископаемого;

### Задание 2 (2 балла)

1. Скачать данные по ссылке

<https://www.kaggle.com/datasets/martaarroyo/palmer-penguins-for-binary-classification>;

2. Построить модель логистической регрессии по столбцу species;
3. Визуализировать полученное разделение данных;

### Задание\* (4 балла)

1. Скачать данные по ссылке

<https://www.kaggle.com/datasets/yasserh/song-popularity-dataset>

2. Проверить данные на мультиколлинеарность: вывести матрицу корреляции и рассчитать VIF;

3. Построить регрессионные модели с зависимой переменной song\_popularity (например, можно взять следующие модели: множественная линейная регрессия, ридж-регрессия, случайный лес. Подробнее: <https://scikit-learn.org/stable/>);

4. Сравнить метрики качества регрессионных моделей, выбрать лучшую, сделать выводы.

Результаты представить в формате .ipynb.